

Module 9 - Architecture Design

TASK
CHECKLIST

- ☒ Select Design Strategy.
- ☐ Design Architecture.
- ☐ Select Hardware and Software.
- ☐ Develop Use Scenarios.
- ☐ Design Interface Structure.
- ☐ Develop Interface Standards.
- ☐ Design Interface Prototype.
- ☐ Evaluate User Interface.
- ☐ Design User Interface.
- ☐ Develop Physical Data Flow Diagrams.
- ☐ Develop Program Structure Charts.
- ☐ Develop Program Specifications.
- ☐ Select Data Storage Format.
- ☐ Develop Physical Entity Relationship Diagram.
- ☐ Denormalize Entity Relationship Diagram.
- ☐ Performance Tune Data Storage.
- ☐ Size Data Storage.

System Architecture

- Describes the system's hardware, software and network environment
 - A system has components
 - System components are usually spread across two or more computers.
 - The architecture would define how the components will be distributed across multiple computers
- Flows primarily from the nonfunctional requirements.
 - General non-functional requirements are refined into more specific requirements
 - These requirements then affect the architecture design

Components of an information system

- Software
- Hardware

Objective of architecture design: to determine **how the software components will be assigned to the hardware devices.**

Components of an information system

- Software
 - What are the major functions of software?
 - How can the software be divided into different parts?
- Hardware
 - What are major types of hardware onto which software can be placed?

Software

Four basic functions:

1. Data storage
2. Data access logic
3. Application logic
4. Presentation logic

Hardware

Three primary components:

1. Client computers
2. Servers
3. Network

Architecture: How the software components of the information system will be assigned to the hardware devices of the system. I.e. **Where each piece of software will be placed.**

Client-Server Architectures

Client-Server Architecture

- The client side is responsible for presentation
- The server is responsible for data storage and data access
- The application logic may reside on:
 - the client
 - the server
 - or split between both
- **Note:** there can be multiple clients or multiple servers.

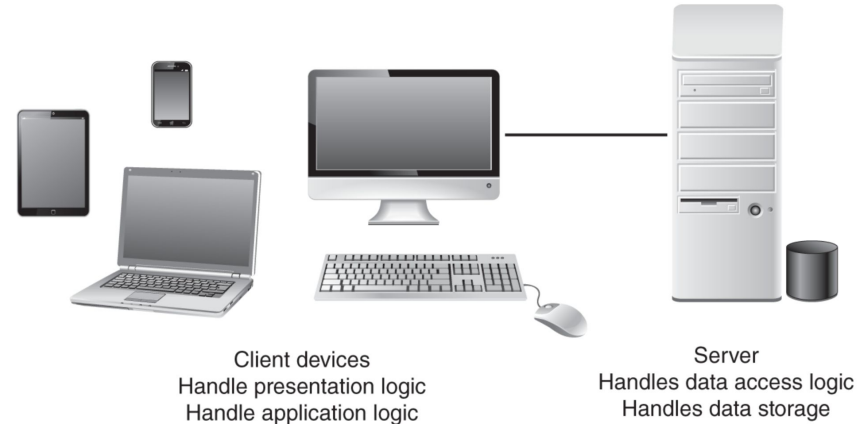


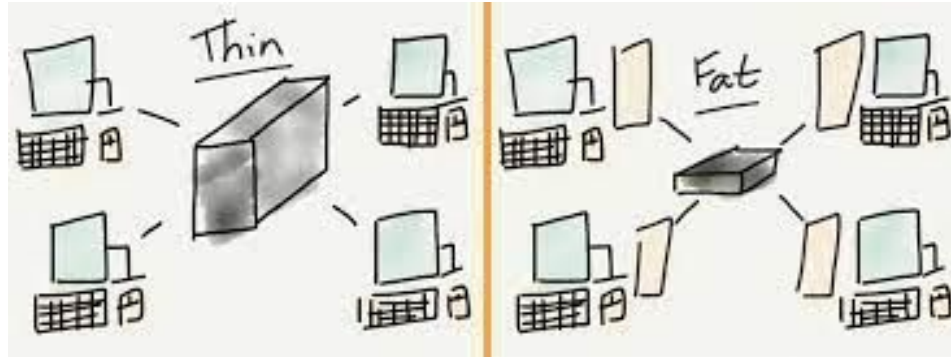
FIGURE 7-1 Two-tiered client-server architecture.

Thick and thin clients

Thick Client: contains all or most of the application logic.

Thin Client: contain just a small portion of the application logic:

- Lower overhead
 - Easier maintenance
- ➡ popular



Benefits of client-server architecture

1. **Scalable**
2. Can **support** many different types of clients and servers by means of **middleware**.
3. It is possible to **design** each of presentation, application, data storage and data access **logics independently**.
4. If a server fails, only the applications requiring that server will fail. The **failed server can be swapped** and replaced.

Client-server architectures are **more complex than all-in-one software**: they have **two parts** -- the software on the client side and the software on the server side. Writing and updating such a system is more complicated.

Client-server tiers

There are many ways the application logic can be partitioned between the client and the server.

- Two-tiered architecture
- Three tiered architecture
- N-tier architecture

Two-tiered architecture

- A common configuration
- Client is responsible for the presentation *and* application logic
- Server is responsible for the data (both storage and access)

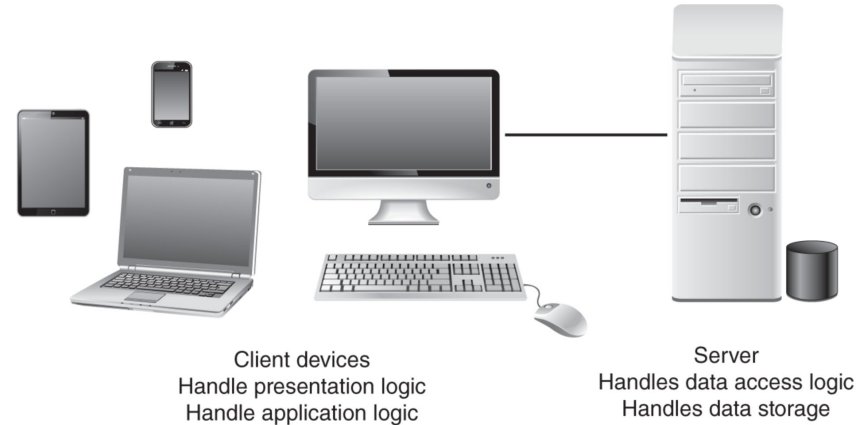
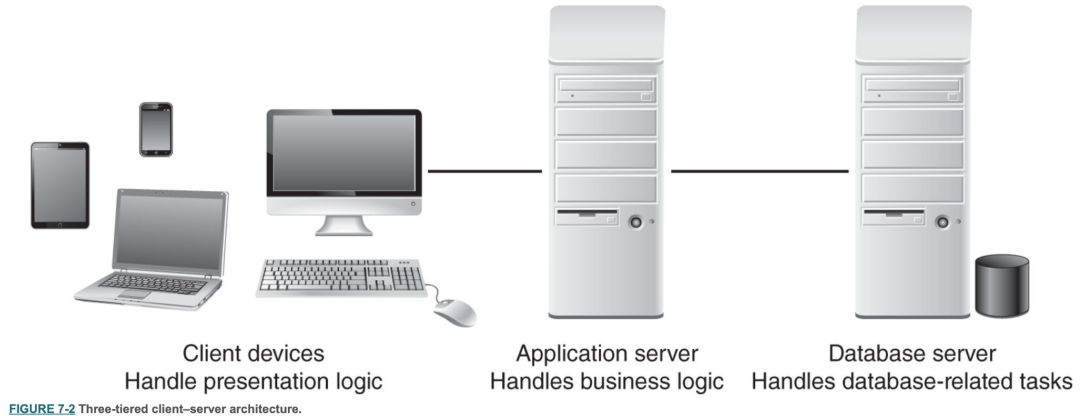


FIGURE 7-1 Two-tiered client-server architecture.

Three-tiered architecture

- Uses three sets of computers
 - **Client**
 - **Application Server**
 - **Database Server**
- **Note:** The **middle tier** (application Server) may be divided into tiers itself
→ **n-tier architecture**



n-tier architecture

- Distributes the work of the application among **multiple specialized servers**.
- Example: A web-based e-commerce system
 - **Web server:** Enables the user to **view** merchandise for sale (responds with HTML documents)
 - **Application server:** enables **detailed processing** such as: *putting items in shopping cart, determining item pricing and availability, computing purchase costs, sales tax, and shipping costs, authorizing payment, etc.*
 - These **modular business logic** that are separately stored on the application servers **can be accessed by any application**. E.g. the cash register application too!

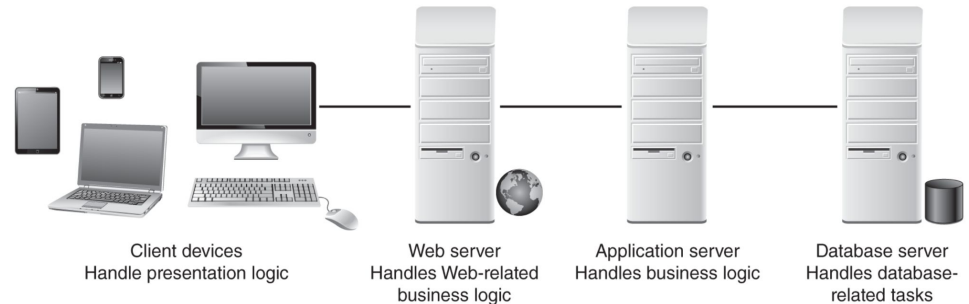


FIGURE 7-3 n-Tiered client-server architecture.

The **primary advantage** of n-tier architecture is that it is **more scalable**: it separates out the processing to better balance the load on different servers.

But n-tier architecture needs a **higher capacity network** and is **much more difficult to program and test**.

Server-based architecture

Server-based architecture

- The server (usually, a central **mainframe computer**) performs all four application functions.
- The clients (“**dumb**” **terminals**) enable users to send and receive messages to and from the the server
- The very first computing architectures were server-based
- They are less common today but still important specially where systems process **very high transaction volume** and **strong security** is required.

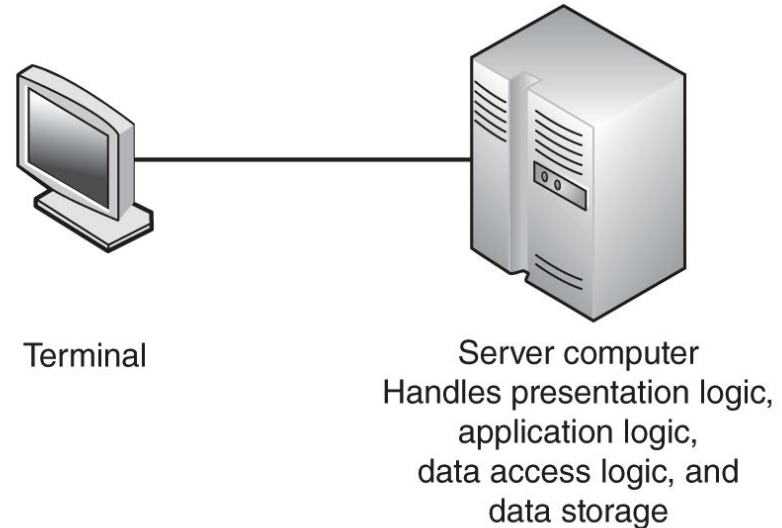


FIGURE 7-4 Server-based architecture.

When a server becomes **overloaded** and unable to quickly process all the user's demands, It should be upgraded. In server-based architecture **increased capacity came only in large, expensive chunks.**

Zero client (ultrathin client)

- A server-based computing model often used today in a virtual desktop infrastructure (VDI)
 - **Zero client device:** a small box that connects a keyboard, mouse, monitor and ethernet connection to a remote server
 - **The server hosts everything:** the client's operating system and all software applications
- Benefits:
 - Power usage is significantly lower than fat client configurations
 - No software at the client device → no vulnerability to malware
 - Administration is easy
 - Multiple virtual PCs can be run on the server hardware reducing the number of physical PCs
 - Limits nonbusiness use of the client computer.
 - An efficient and secure way to deliver applications to end users

Mobile Application Architecture

Mobile Application Architecture

- Mobile devices include tablets, smartphones and other portable devices
- **Rich Client:**
 - Application requires local processing using the device resources (e.g. camera or GPS)
 - The device is only occasionally connected to the server
 - Business, data access and presentation logic reside on the device
- **Thin Web-Based Client:**
 - The application can rely on the availability of server processing
 - The device will always be connected
 - Business and data access logic will reside on the server
- **Rich Internet Application (RIA):**
 - Application requires a rich User Interface (UI)
 - Only limited access to local device resources
 - Must be portable to other platforms

Rich Internet Applications are web-based applications that function as traditional desktop applications; However **web browsers are required for access and software installation on the mobile device is not required.**

Development options

- **Native applications**
 - Run on a specific device with a specific operating system
 - Provide the richest user experience and can take full advantage of the device's resources
 - Must be rebuilt for each native operating system
 - Must be updated as new device models and operating system versions are released.
- **Cross-platform frameworks**
 - Create a mobile app in Web-based technologies like Javascript or HTML and use the framework to adapt the application for multiple devices
- **Mobile Web apps**
 - Platform independent
 - Built with web-technology
 - Cannot use the device's hardware or software
 - Requires internet connection to work
 - Will provide the most generic user experience

Advances in Architecture Configurations

Virtualization

- Creation of a virtual device or resource, such as a server or storage device
- **Server Virtualization:**
 - Partitioning a physical server into smaller virtual servers that are independent of each other, run their own OS and can be rebooted independently
 - Less hardware is required → reduced costs.
- **Storage Virtualization:**
 - Combining multiple network storage devices into what appears to be a single storage unit.
 - The storage devices are connected in a storage area network.
 - Backup, archiving and recovery are easier and faster
- Other types of virtualization: **application virtualization** and **virtual desktop infrastructure**.

Cloud Computing

- **Cloud:** a set of hardware, networks, storage, services, and interfaces that combine to deliver **aspects of computing as a service**
- Implemented as:
 - **Public cloud**
 - Services are provided “as a service” **over the internet** with little or no control over the underlying technology infrastructure
 - **Private cloud**
 - Services are deployed over a company intranet or hosted data center
 - **Hybrid cloud**
- **Benefits:** scalability, cost reduction, device independence, performance

Cloud computing suppliers utilize **virtualization as a key enabling technology.**

Creating an Architecture Design

Often, the **existing infrastructure** in the organization limits the choice of architecture.

Each of the architectures has its **strength and weaknesses**.

The **first step** in creating an architecture design is to **refine the nonfunctional requirements**.

Operational Requirements

Type of Requirement	Definition	Examples
Technical Environment Requirements	Special hardware, software, and network requirements imposed by business requirements	• The system will work over the Web environment with any browser. • All office locations will have an always-on network connection to enable real-time database updates. • A version of the system will be provided for customers connecting over the Internet via a small-screen smartphone.
System Integration Requirements	The extent to which the system will operate with other systems	• The system must be able to import and export Excel spreadsheets. • The system will read and write to the main inventory database in the inventory system.
Portability Requirements	The extent to which the system will need to operate in other environments	• The system must be able to work with different operating systems (i.e., Linux, Windows 10). • The system may need to operate with mobile devices such as an iPad.
Maintainability Requirements	Expected business changes to which the system should be able to adapt	• The system will be able to support more than one manufacturing plant upon 3 months advance notice. • New versions of the system will be released every 2 months.

FIGURE 7-5 Operational requirements.

Performance Requirements

Type of Requirement	Definition	Examples
Speed Requirements	The time within which the system must perform its functions	• Response time must be 4 seconds or less for any transaction over the network. • The inventory database must be updated in real time. • Orders will be transmitted to the factory floor every 3 minutes.
Capacity Requirements	The total and peak number of users and the volume of data expected	• There will be a maximum of 2000 simultaneous users at peak use times. • A typical transaction will require the transmission of 300 K of data. • The system will store data on approximately 50,000 customers for a total of about 2 GB of data.
Availability and Reliability Requirements	The extent to which the system will be available to the users and the permissible failure rate due to errors	• The system should be available 24/7, with the exception of scheduled maintenance. • Scheduled maintenance shall not exceed one 6-hour period each month. • The system will have 99% uptime performance.

FIGURE 7-6 Performance requirements.

The **testability** of performance requirements is a key issue. Each requirement must be **measurable**.

Speed requirements are two-fold:

- How fast the system must operate (**response time**)
- How long it takes transactions in one part of the system to be **reflected in other parts**.

Security Requirements

Security is the ability to protect the information system from **disruption and data loss**, whether caused **by an intentional act or a random event**.

Although security is primarily the responsibility of the operations group, nonetheless, developers must ensure the **security within the information system** itself.

Historically, the greatest security threat has come from inside the organization itself.

Security Requirements

Type of Requirement	Definition	Examples
System Value Estimates	Estimated business value of the system and its data	• The system is not mission critical, but a system outage is estimated to cost \$150,000 per hour in lost revenue. • A complete loss of all system data is estimated to cost \$20 million.
Access Control Requirements	Limitations on who can access what data	• Only department managers will be able to change inventory items within their own department. • Customer Service personnel will be able to read and create items in the customer file, but cannot change or delete items.
Encryption and Authentication Requirements	Defines what data will be encrypted where and whether authentication will be needed for user access	• Data will be encrypted from the user's computer to the website to provide secure ordering. • Users logging in from outside the office will be required to authenticate.
Virus Control Requirements	Controls the spread of viruses	• All uploaded files will be checked for viruses before being saved in the system.

FIGURE 7-7 Security requirements.

The **most important asset** in any organization is not the equipment; it is the **data**.

A **mission critical system** is literally critical to the survival of the organization. They cannot be permitted to fail.

One of the best ways to prevent unauthorized access to data is **encryption**: disguising information by the use of mathematical algorithms (or formulas).

The single most common security problem comes from **viruses**.

Cultural and Political Requirements

Type of Requirement	Definition	Examples
Multilingual Requirements	The language in which the system will need to operate	• The system will operate in English, French, and Spanish. • Country managers will be able to define new fields in the product database to capture country-specific information. • Country managers will be able to change the format of the telephone-number field in the customer database. • All date fields will be explicitly identified as using the month-day-year format. • All weight fields will be explicitly identified as being stated in kilograms.
Customization Requirements	Specification of what aspects of the system can be changed by local users	
Making Unstated Norms Explicit	Explicitly stating assumptions that differ from country to country	
Legal Requirements	The laws and regulations that impose requirements on the system	• Personal information about customers cannot be transferred out of European Union countries into the United States. • It is against US federal law to divulge information on who rented what videotape, so access to a customer's rental history is permitted only to regional managers.

FIGURE 7-8 Cultural and political requirements.

One of the most challenging aspects in designing multilingual systems is **getting a good translation**. The other challenge often is **screen space**.

Customization often makes it more difficult to create and maintain the application.

Designing the Architecture

Your Turn!

- ❑ Reading: Designing the Architecture
- ❑ Reading: Hardware and Software Specification
- ❑ Applying the Concepts at DronTeg

Requirements	Server-Based	Thin Client-Server	Thick Client-Server
Operational Requirements			
System Integration Requirements	✓	✓	✓
Portability Requirements		✓	
Maintainability Requirements	✓	✓	
Performance Requirements			
Speed Requirements		✓	✓
Capacity Requirements		✓	✓
Availability/Reliability Requirements	✓	✓	✓
Security Requirements			
High System Value	✓	✓	
Access Control Requirements	✓		
Encryption/Authentication Requirements		✓	✓
Virus Control Requirements	✓		
Cultural/Political Requirements			
Multilingual Requirements		✓	
Customization Requirements		✓	
Making Unstated Norms Explicit		✓	
Legal Requirements	✓	✓	✓

FIGURE 7-9 Nonfunctional requirements and their implications for architecture design.